

Logistic regression

Classification: K-Nearest Neighbor, Naïve Bayes,

Logistic Regression

Linear Regression

1. Home prices
2. Weather
3. Stock price

Predicted value is
continuous

Classification

1. Email is spam or not
2. Will customer buy life insurance?
3. Which party a person is going to vote for?
 1. Democratic
 2. Republican
 3. Independent

Predicted value is
categorical



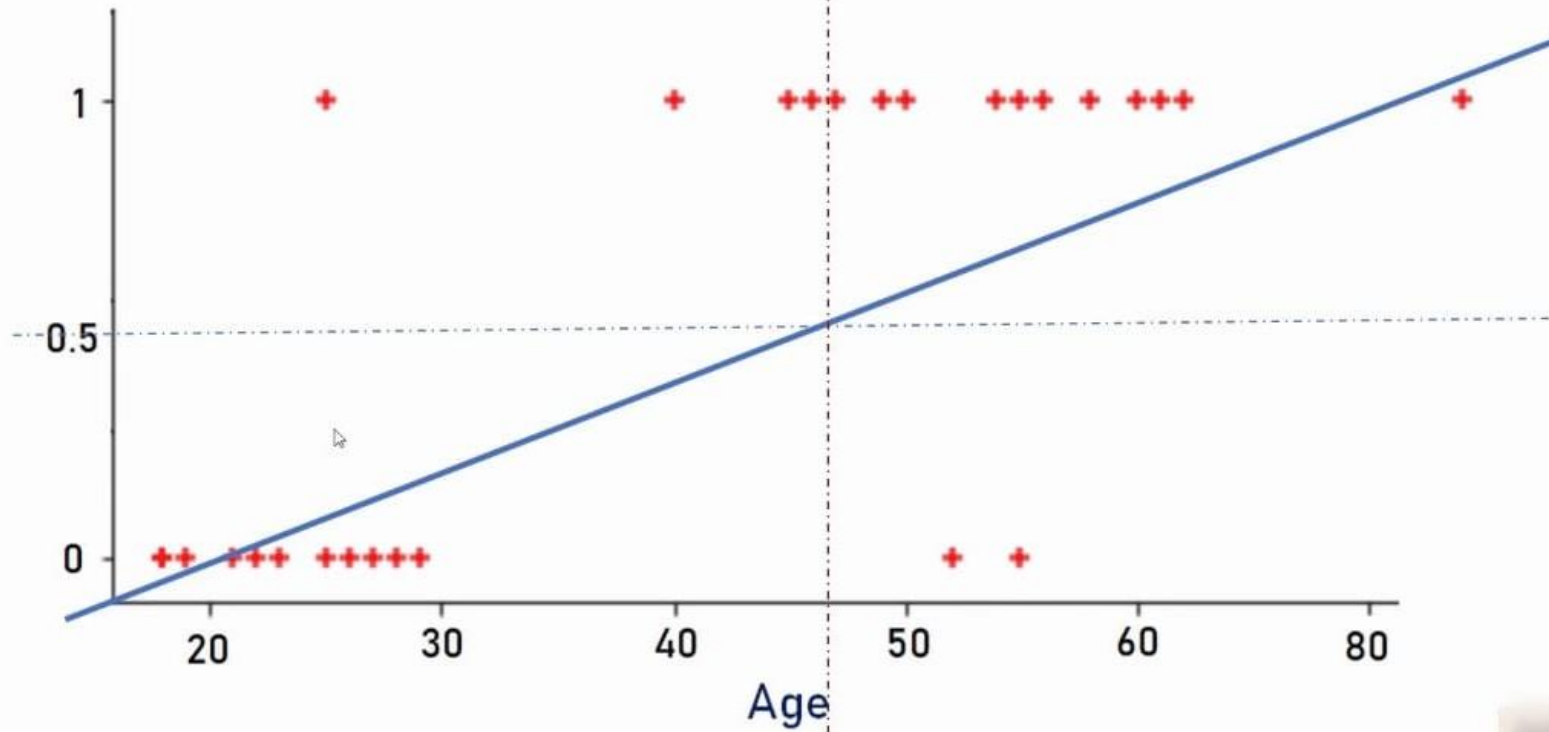
- Logistic regression model transforms the linear regression function continuous value output into categorical value output using a sigmoid function which maps any real-valued set of independent variables input into a value between 0 and 1
- This function is known as the logistic function.

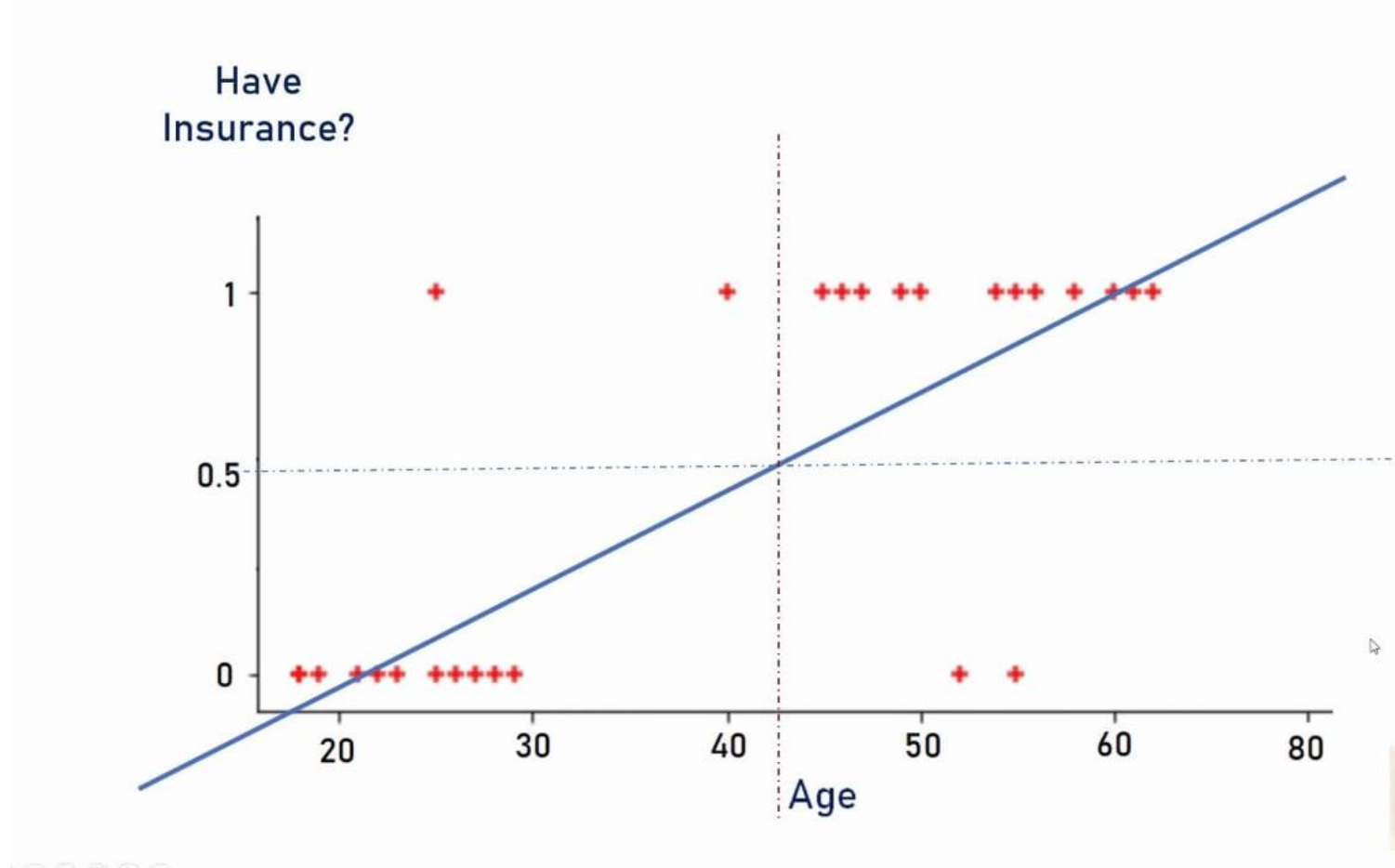


age	have_insurance
22	0
25	0
47	1
52	0
46	1
56	1
55	0
60	1
62	1
61	1
18	0
28	0
27	0
29	0
49	1

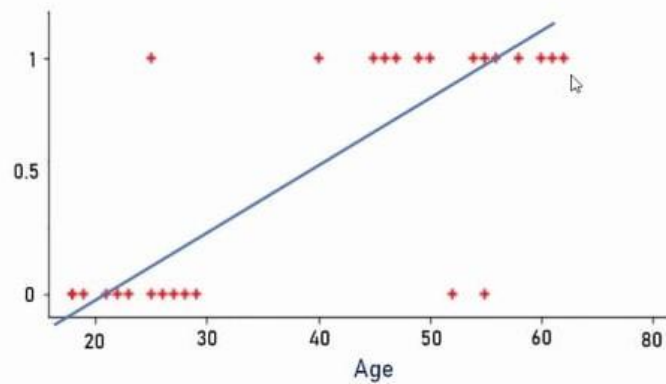


Have
Insurance?

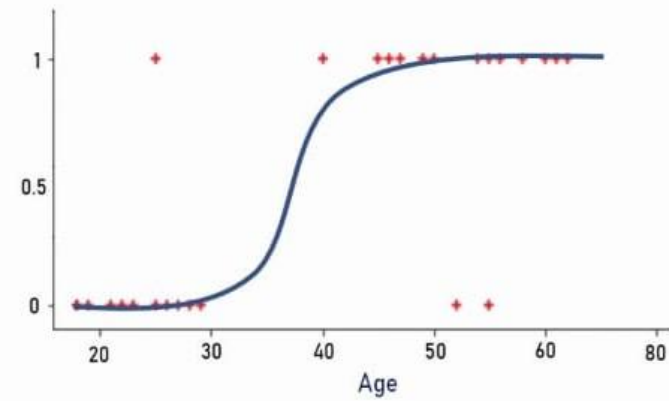




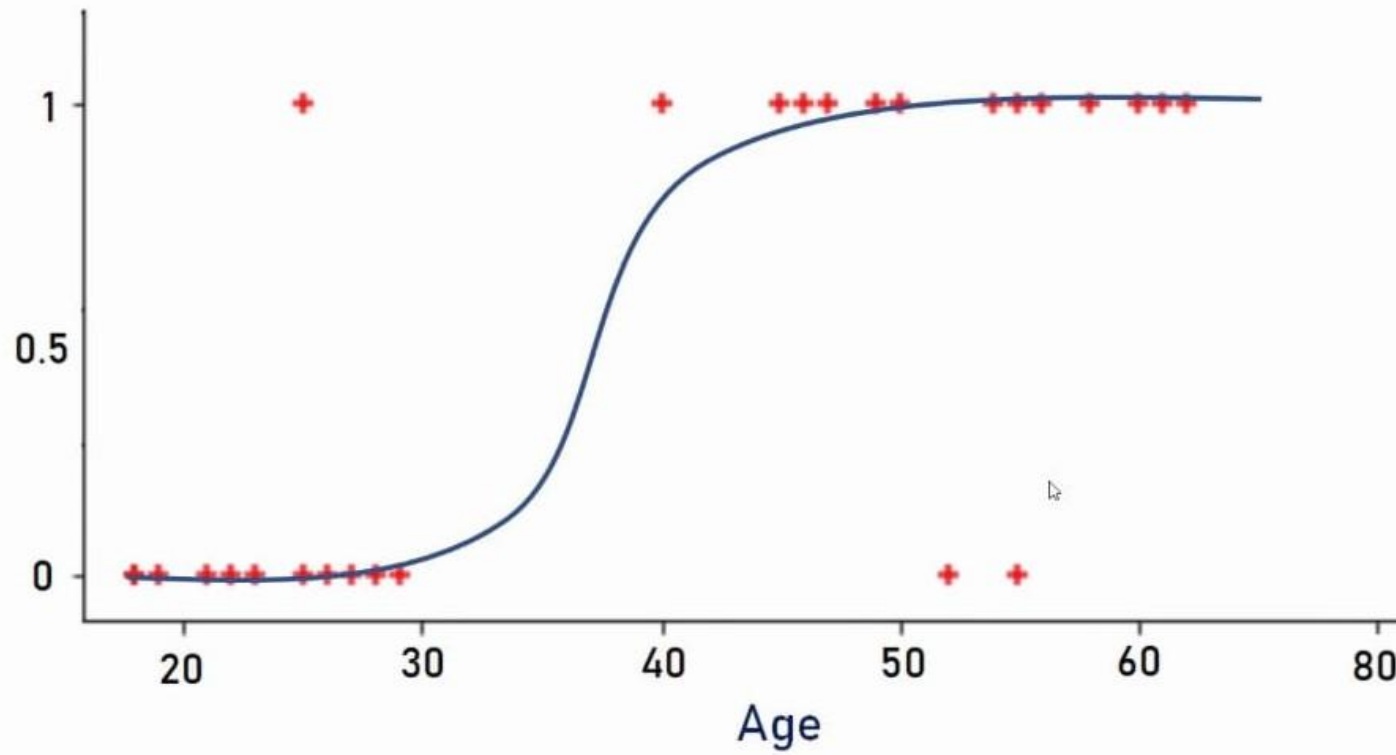
$$y = m * x + b$$



$$y = \frac{1}{1 + e^{-(m*x+b)}}$$



Have
Insurance?



Suppose input features are represented as a matrix:

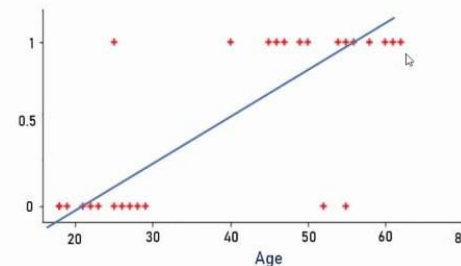
$$X = \begin{bmatrix} x_{11} & \dots & x_{1m} \\ x_{21} & \dots & x_{2m} \\ \vdots & \ddots & \vdots \\ x_{n1} & \dots & x_{nm} \end{bmatrix}$$

and the dependent variable is Y having only binary value i.

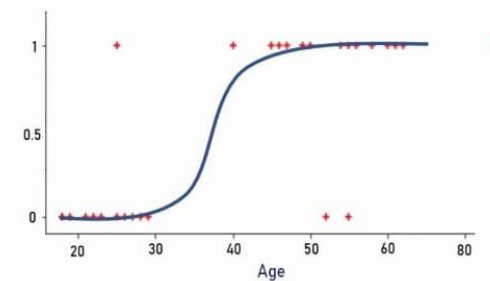
$$Y = \begin{cases} 0 & \text{if Class 1} \\ 1 & \text{if Class 2} \end{cases}$$

$$y = \frac{1}{1 + e^{-z}}$$

$$y = m * x + b$$



$$y = \frac{1}{1 + e^{-(m*x+b)}}$$



Log-Loss for Class 1 ($y = 1$)

$$\text{Log-Loss} = -\log(p)$$

p = predicted probability that the sample belongs to **Class 1**

If $p \rightarrow 1 \rightarrow$ loss is **small**

If $p \rightarrow 0 \rightarrow$ loss is **very large**

Log-Loss for Class 0 ($y = 0$)

$$\text{Log-Loss} = -\log(1 - p)$$

Meaning:

p = predicted probability of **Class 1**

If $p \rightarrow 0 \rightarrow$ loss is **small**

If $p \rightarrow 1 \rightarrow$ loss is **very large**

Logarithm strongly penalizes confident wrong predictions.



- Refer below link for diagrammatic representation. [Logistic Regression](#)

A suitable loss function in logistic regression is called the Log-Loss, or binary cross-entropy. This function is:

$$\text{Log-Loss} = \sum_{i=0}^n -(y_i * \log(p_i) + (1 - y_i) * \log(1 - p_i))$$

- If $y=1 \rightarrow$ first ON, second OFF
- If $y=0 \rightarrow$ first OFF, second ON

Symbol	Meaning
(n)	Total number of data samples
(i)	Index of a data point
(y_i)	True label (0 or 1)
(p_i)	Predicted probability that ($y_i = 1$)
(log)	Natural logarithm



Log-loss measures the error of a classification model by comparing predicted probabilities with actual class labels, **penalizing confident wrong predictions** more heavily.

Case 1: When the true label is 1 ($y_i = 1$)

$$\text{Loss} = -\log(p_i)$$

- If p_i is close to 1 $\rightarrow$ loss is **small**
- If p_i is close to 0 $\rightarrow$ loss is **very large**

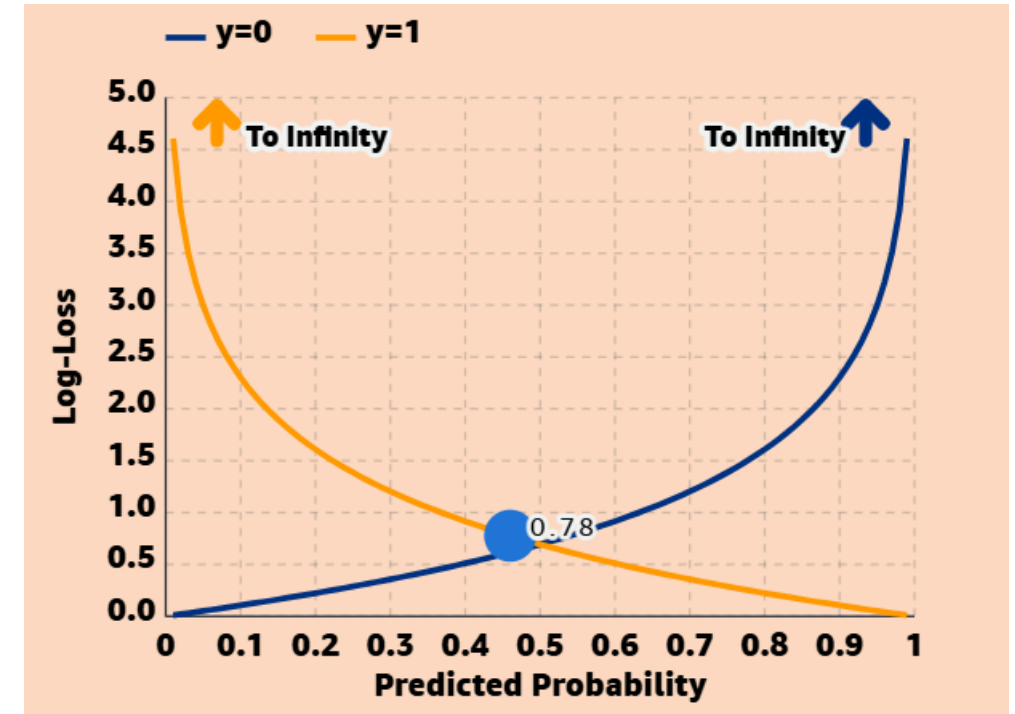
Case 2: When the true label is 0 ($y_i = 0$)

$$\text{Loss} = -\log(1 - p_i)$$

- If p_i is close to 0 $\rightarrow$ loss is **small**
- If p_i is close to 1 $\rightarrow$ loss is **very large**

Why Log-Loss is used in Logistic Regression

- Penalizes **wrong predictions** heavily
- Encourages **confident and correct probability estimates**
- Smooth and differentiable $\rightarrow$ suitable for **gradient descent**



Log-Loss (Binary Cross-Entropy) Formula=
entropy is the machine learning metric that
measures the unpredictability or impurity in
the system.

K-Nearest Neighbor

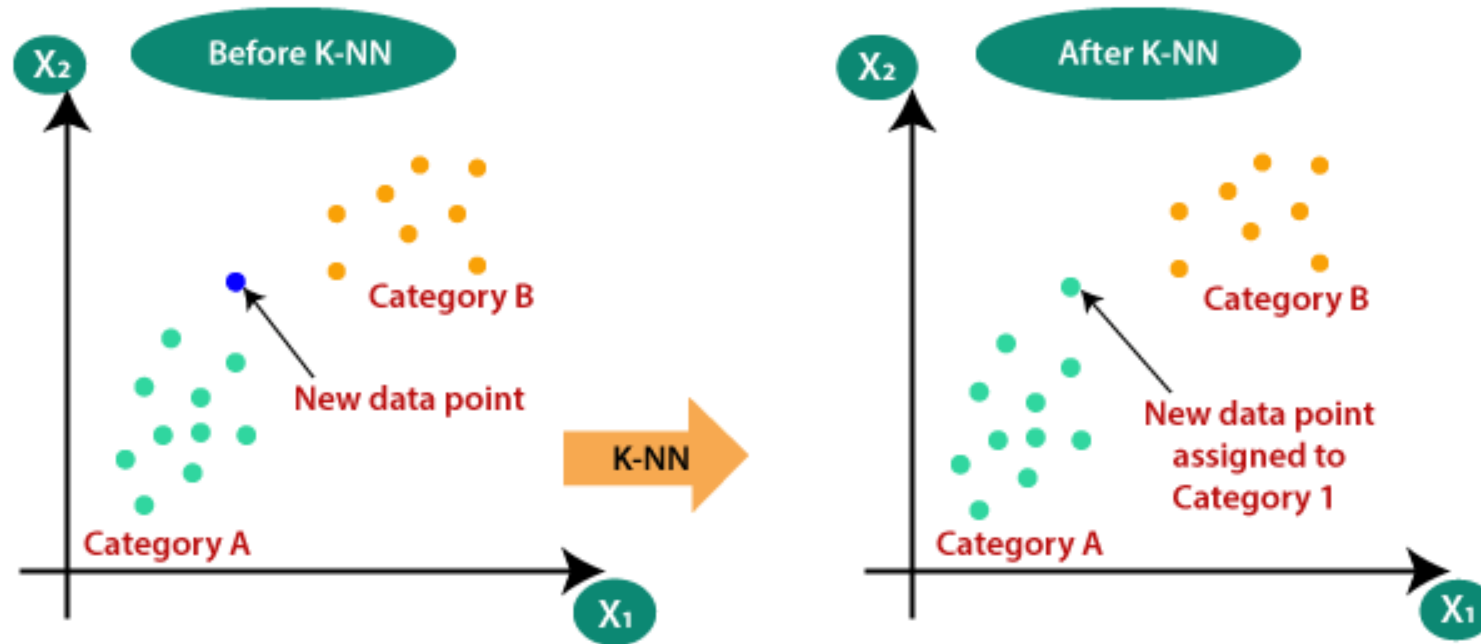
- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on **Supervised Learning technique**.
- K-NN algorithm assumes the similarity between the new data and available data and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.



- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog.
- So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



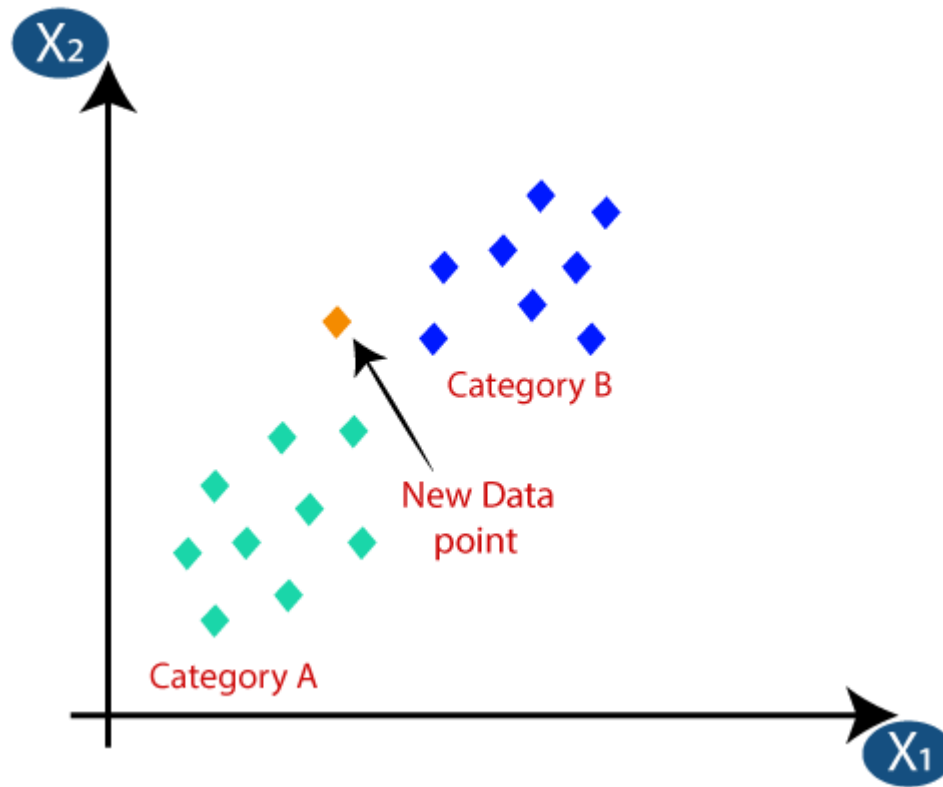
- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



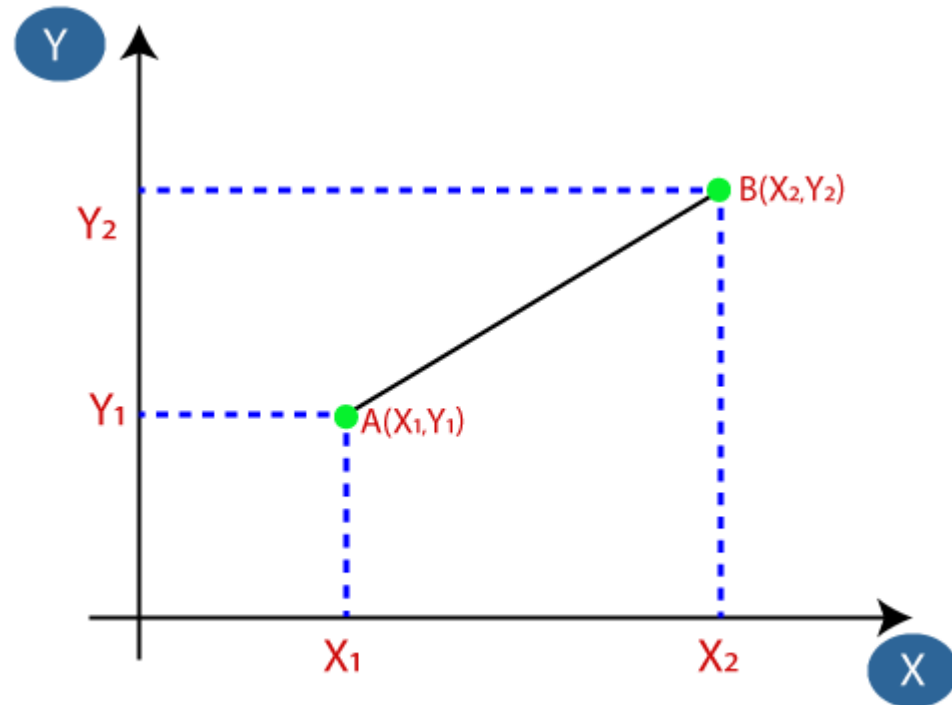
- The K-NN working can be explained on the basis of the below algorithm:
- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.



- Suppose we have a new data point and we need to put it in the required category. Consider the below image



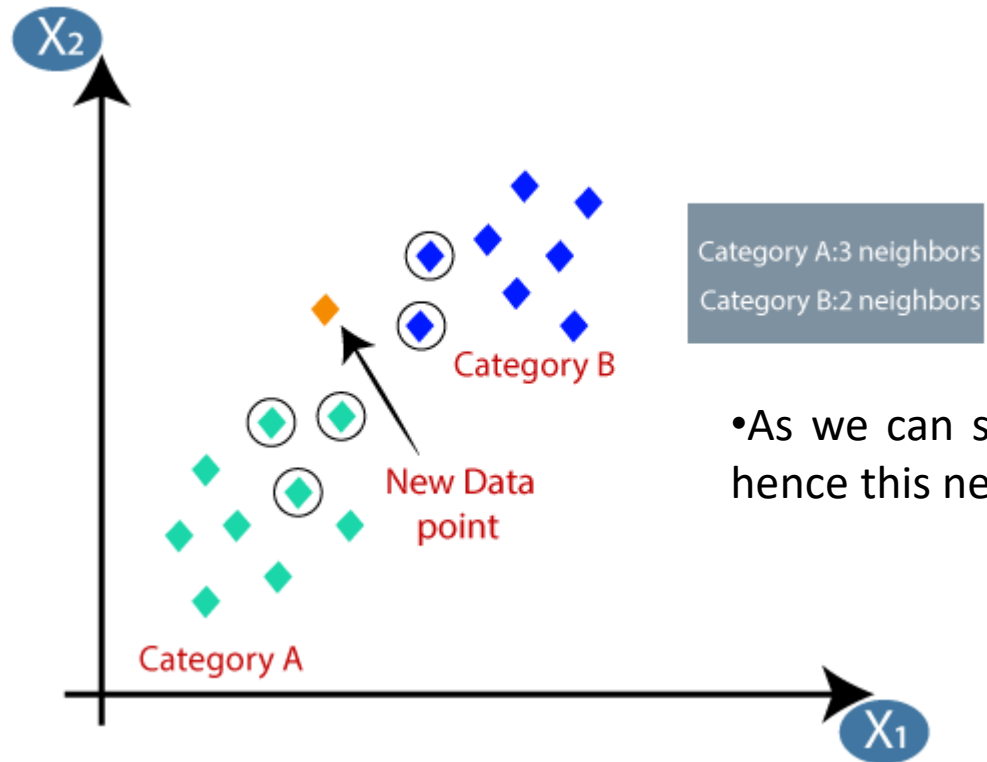
- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:



$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$



- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.



How to select the value of K in the K-NN Algorithm

- Below are some points to remember while selecting the value of K in the K-NN algorithm:
- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as $K=1$ or $K=2$, can be noisy and lead to the effects of outliers in the model.



- **Advantages of KNN Algorithm:**

Simple to use: Easy to understand and implement.

Training phase = storing all labelled data

Few parameters: Only needs to set the number of neighbors (k) and a distance method.

Versatile: Works for both classification and regression problems.

- **Disadvantages of KNN Algorithm:**

Slow with large data: Needs to compare every point during prediction.

Struggles with many features: Accuracy drops when data has too many features.

Can Overfit: It can overfit especially when the data is high-dimensional or not clean.



- **Applications of KNN**
- **Recommendation Systems:** Suggests items like movies or products by finding users with similar preferences.
- **Spam Detection:** Identifies spam emails by comparing new emails to known spam and non-spam examples.
- **Customer Segmentation:** Groups customers by comparing their shopping behavior to others.
- **Speech Recognition:** Matches spoken words to known patterns to convert them into text.



Distance Metrics Used in KNN Algorithm

- KNN uses distance metrics to identify nearest neighbor, these neighbors are used for classification and regression task. To identify nearest neighbor, we use below distance metrics:

Euclidean Distance

- Euclidean distance is defined as the straight-line distance between two points in a plane or space. You can think of it like the shortest path you would walk if you were to go directly from one point to another.

The formula for Euclidean distance between two points

For two points A and B in an n -dimensional space:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Where:

- $A = (A_1, A_2, \dots, A_n)$
- $B = (B_1, B_2, \dots, B_n)$
- $d(A, B)$ represents the Euclidean distance between A and B .

For 2D space ($n = 2$):

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

For 3D space ($n = 3$):

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

- **Manhattan Distance**

This is the total distance you would travel if you could only move along horizontal and vertical lines like a grid or city streets. It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

$$d = |x_i - y_i|$$

- **Minkowski Distance**

- Minkowski distance is like a family of distances, which includes both Euclidean and Manhattan distances as special cases.

$$d = \left(\sum_{i=1}^n (x_i - y_i)^p \right)^{\frac{1}{p}}$$

From the formula above, when $p=2$, it becomes the same as the Euclidean distance formula and when $p=1$, it turns into the Manhattan distance formula. Minkowski distance is essentially a flexible formula that can represent either Euclidean or Manhattan distance depending on the value of p .



Naïve Bayes

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- Make a naïve
(Not having experienced or been subjected to something)
assumption that each and every features are independent of each other.

If x_1, x_2, x_3 are three independent features and y is the dependent feature

$$P(y|x_1, x_2, x_3) = \frac{P(x_1, x_2, x_3|y) * P(y)}{P(x_1, x_2, x_3)}$$



Where,

P(A|B) is Posterior probability:

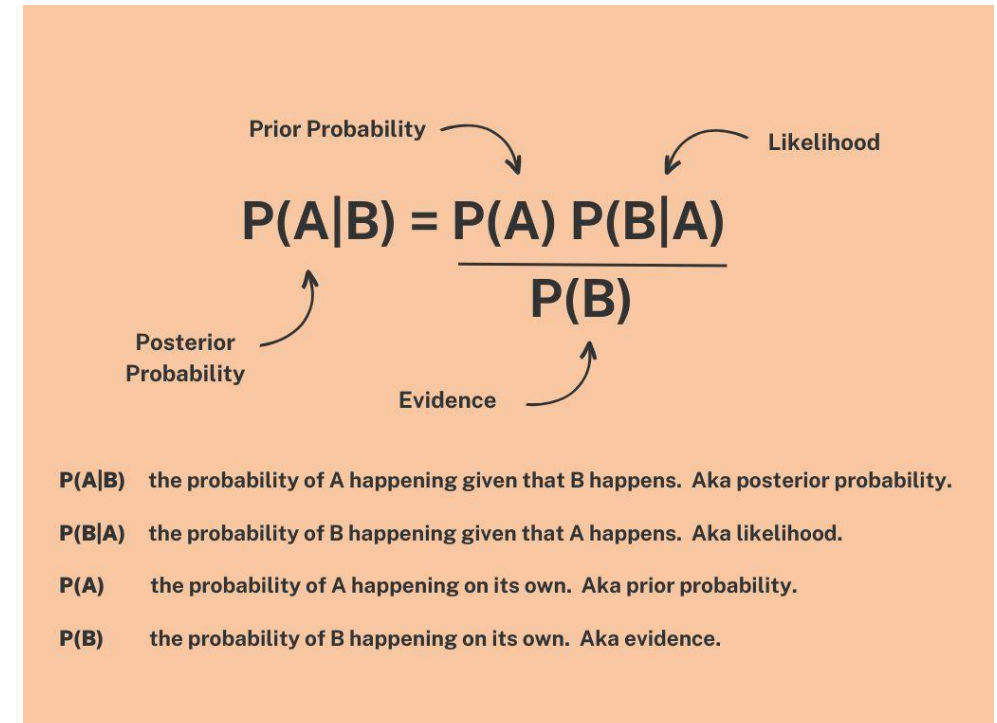
Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability:

Probability of the evidence given that the probability of a hypothesis is true.

P(A) is Prior Probability: Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability: Probability of Evidence.



Passenger Id	Name	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Survived
1	Braund, Mr. Owen Harris	3	male	22	1	0	21171	7.25		S	0
2	Cumings, Mrs. John Bradley	1	female	38	1	0	17599	71.2833	C85	C	1
3	Heikkinen, Miss. Laina	3	female	26	0	0	3101282	7.925		S	1
4	Futrelle, Mrs. Jacques Heath	1	female	35	1	0	113803	53.1	C123	S	1
5	Allen, Mr. William Henry	3	male	35	0	0	373450	8.05		S	0
6	Moran, Mr. James	3	male		0	0	330877	8.4583		Q	0
7	McCarthy, Mr. Timothy J	1	male	54	0	0	17463	51.8625	E46	S	0
8	Palsson, Master. Gosta Leonard	3	male	2	3	1	349909	21.075		S	0
9	Johnson, Mrs. Oscar	3	female	27	0	2	347742	11.1333		S	1
10	Nasser, Mrs. Nicholas	2	female	14	1	0	237736	30.0708		C	1

$$P \left(\frac{\text{Survived}}{\text{Male \& Class \& Age \& Cabin \& Fare}} \right)$$



Working of Naïve Bayes' Classifier:

- Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**".
- So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions.
- Convert the given dataset into frequency tables.
- Generate Likelihood table by finding the probabilities of given features.
- Now, use Bayes theorem to calculate the posterior probability



Problem 1: if weather is sunny player should play or not?

S.No	Weather	Play tennis
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes



Step 1: Convert the data into frequency table

Weather	Yes	No	Total
Overcast	5	0	5
Rainy	2	2	4
Sunny	3	2	5
Total	10	4	14

Step 2: Generate likelihood table by using probability of given features

Weather	Yes	No	
Overcast	5	0	$P(\text{overcast}) = 5/14 = 0.35$
Rainy	2	2	Prior Probabilities $P(\text{Rainy}) = 4/14 = 0.29$
Sunny	3	2	$P(\text{Sunny}) = 5/14 = 0.35$
Total	10	4	$P(\text{Yes}) = 10/14 = 0.71$ and $P(\text{No}) = 4/14 = 0.29$



Step 3: To Find Posterior Probability

- **Bayes' Theorem**

- $P(A | B) = \frac{P(B|A) P(A)}{P(B)}$

Find:

P(Yes|Sunny)

$$P(\text{Yes} | \text{Sunny}) = \frac{P(\text{Sunny} | \text{Yes}) P(\text{Yes})}{P(\text{Sunny})}$$

$$P(\text{Yes} | \text{Sunny}) = \frac{0.3 \times 0.71}{0.35}$$

$$= \frac{0.213}{0.35}$$

$$= 0.61$$

- $P(\text{Sunny} | \text{Yes}) = \frac{3}{10} P(\text{Sunny} | \text{No}) = \frac{2}{4}$

$$P(\text{No} | \text{Sunny}) = \frac{0.5 \times 0.29}{0.35}$$

$$= \frac{0.145}{0.35}$$

$$= 0.41$$

Class

Play = Yes

Play = No

Posterior Probability

0.61

0.41

$P(\text{Yes}|\text{Sunny}) > P(\text{No}|\text{Sunny})$; So player can play the game



- Types of Naïve Bayes Model:
- **Gaussian:** The Gaussian model assumes that features follow a normal distribution.
- if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.
- **Multinomial:** The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed.
- It is primarily used for document classification problems-document belongs to which category such as Sports, Politics, education, etc.
The classifier uses the frequency of words for the predictors.
- **Bernoulli:** The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables.
- Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.



- Data Pre-processing step:

```
import numpy as nm
```

```
import matplotlib.pyplot as mtp
```

```
import pandas as pd
```

```
# Importing the dataset
```

```
dataset = pd.read_csv('user_data.csv')
```

```
x = dataset.iloc[:, [2, 3]].values
```

```
y = dataset.iloc[:, 4].values
```

```
# Splitting the dataset into the Training set and Test set
```

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.25, random_state = 0)
```

```
# Feature Scaling
```

```
from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()
```

```
x_train = sc.fit_transform(x_train)
```

```
x_test = sc.transform(x_test)
```



Index	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
10	15570769	Female	26	80000	0
11	15606274	Female	26	52000	0
12	15746139	Male	20	86000	0
13	15704987	Male	32	18000	0
14	15628972	Male	18	82000	0
15	15697686	Male	29	80000	0
16	15733883	Male	47	25000	1
17	15617482	Male	45	26000	1
18	15704583	Male	46	28000	1
19	15621083	Female	48	29000	1

- # Fitting Naive Bayes to the Training set

```
from sklearn.naive_bayes import GaussianNB  
classifier = GaussianNB()  
classifier.fit(x_train, y_train)  
Prediction of the test set result:  
# Predicting the Test set results  
y_pred = classifier.predict(x_test)
```

	0
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	0
9	1
10	0
11	0
12	0
13	0

	0
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	0
9	0
10	0
11	0
12	0
13	0



- # Making the Confusion Matrix
- **from** sklearn.metrics **import** confusion_matrix
- `cm = confusion_matrix(y_test, y_pred)`

